LAB 1 : WRITE A C PROGRAM THAT CONTAINS A STRING (CHAR POINTER) \HELLO WORLD. THE PROGRAM SHOULD XOR EACH CHARACTER IN THIS STRING WITH ZERO AND DISPLAY THE RESULT

```
#include <stdio.h>

int main() {
   char *str = "Hello World";
   int i = 0;
   printf("Original string: %s\n", str);
   printf("After XOR with 0: ");
  while(str[i]!= '\0') {
      char result = str[i] ^ 0;
      printf("%c", result);
      i++;
   }
   printf("\n");
   return 0;
}
```

lab 2: WRITE A C PROGRAM THAT CONTAINS A STRING (CHAR POINTER) \HELLO WORLD. THE PROGRAM SHOULD AND & XOR EACH CHARACTER IN THIS STRING WITH 127 AND DISPLAY THE RESULT

```
#include <stdio.h>


int main() {
   char *str = "Hello World";
```

```
    printf("Original string: %s\n", str);
    // Using AND with 127
    printf("After AND with 127:\n");
    for (int i = 0; str[i]!= '\0'; i++) {
        char result = str[i] & 127;
        printf("%c", result);
    }
    printf("\n\n");
    // Using XOR with 127
    printf("After XOR with 127:\n");
    for (int i = 0; str[i]!= '\0'; i++) {
        char result = str[i] ^ 127;
        printf("%c", result);
    }
    printf("\n");
    return 0;
}
```

lab 3 (a): WRITE A JAVA PROGRAM TO PERFORM ENCRYPTION AND DECRYPTION USING THE CAESAR CIPHER

```
import java.util.Scanner;

public class SimpleCaesar {
```

```
// Shift one character
static char shiftChar(char ch, int shift) {
    if (ch >= 'A' && ch <= 'Z') {
        return (char) ('A' + (ch - 'A' + shift) % 26);
    }
    if (ch >= 'a' && ch <= 'z') {
        return (char) ('a' + (ch - 'a' + shift) % 26);
    }
    return ch; // don't change spaces, numbers, symbols
}

// Encrypt
static String encrypt(String text, int shift) {
    StringBuilder sb = new StringBuilder();
    for (char ch : text.toCharArray()) {
        sb.append(shiftChar(ch, shift));
    }
    return sb.toString();
}

// Decrypt
static String decrypt(String text, int shift) {
    return encrypt(text, 26 - shift); // just shift backward
}
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");
        String text = sc.nextLine();

        System.out.print("Enter shift: ");
        int shift = sc.nextInt();

        System.out.println("Encrypted: " + encrypt(text, shift));
        System.out.println("Decrypted: " + decrypt(encrypt(text, shift), shift));

        sc.close();
    }
}
```

LAB 3 (b): WRITE A JAVA PROGRAM TO PERFORM ENCRYPTION AND DECRYPTION USING THE SUBSTITUTION CIPHER

```
import java.util.Scanner;

public class SubstitutionCipher {

    // Standard alphabet
    static final String ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```

```
// Substitution key - Must be 26 unique letters
static final String KEY = "QWERTYUIOPASDFGHJKLZXCVBNM";



public static String encrypt(String plainText) {
    plainText = plainText.toUpperCase();
    StringBuilder cipherText = new StringBuilder();

    for (int i = 0; i < plainText.length(); i++) {
        char c = plainText.charAt(i);
        if (Character.isLetter(c)) {
            int index = ALPHABET.indexOf(c);
            char encryptedChar = KEY.charAt(index);
            cipherText.append(encryptedChar);
        } else {
            // Keep spaces and symbols as it is
            cipherText.append(c);
        }
    }
    return cipherText.toString();
}

// Decryption function
public static String decrypt(String cipherText) {
    cipherText = cipherText.toUpperCase();
```

```
    StringBuilder plainText = new StringBuilder();

    for (int i = 0; i < cipherText.length(); i++) {
        char c = cipherText.charAt(i);
        if (Character.isLetter(c)) {
            int index = KEY.indexOf(c);
            char decryptedChar = ALPHABET.charAt(index);
            plainText.append(decryptedChar);
        } else {
            plainText.append(c);
        }
    }
    return plainText.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("--- Substitution Cipher ---");
    System.out.println("Alphabet : " + ALPHABET);
    System.out.println("Key      : " + KEY);
    System.out.print("\nEnter Plain Text: ");
    String plain = sc.nextLine();

    String encrypted = encrypt(plain);
```

```
        System.out.println("Encrypted Text: " + encrypted);

        String decrypted = decrypt(encrypted);
        System.out.println("Decrypted Text: " + decrypted);

        sc.close();
    }
}
```

Lab 3 (c): WRITE A JAVA PROGRAM TO PERFORM ENCRYPTION AND DECRYPTION USING THE HILL CIPHER

```
import java.util.Scanner;

class HillRow {

    static String encrypt(String text, int[][] key) {
        text = text.toUpperCase().replaceAll(" ", "");
        if (text.length() % 2!= 0) text += "X";

        String cipher = "";
        for (int i = 0; i < text.length(); i += 2) {
            int p1 = text.charAt(i) - 'A';
            int p2 = text.charAt(i + 1) - 'A';
```

```
        // Row vector * Matrix: C = P * K
        // C1 = P1*K00 + P2*K10
        // C2 = P1*K01 + P2*K11
        int c1 = (p1 * key[0][0] + p2 * key[1][0]) % 26;
        int c2 = (p1 * key[0][1] + p2 * key[1][1]) % 26;

        cipher += (char) (c1 + 'A');
        cipher += (char) (c2 + 'A');
    }
    return cipher;
}

static String decrypt(String cipher, int[][] key) {
    // Inverse for ROW method
    int det = (key[0][0] * key[1][1] - key[0][1] * key[1][0]) % 26;
    if (det < 0) det += 26;

    int detInv = -1;
    for (int i = 0; i < 26; i++) {
        if ((det * i) % 26 == 1) { detInv = i; break; }
    }
    if (detInv == -1) return "Key not invertible";

    int[][] invKey = new int[2][2];
    invKey[0][0] = (key[1][1] * detInv) % 26;
```

```
    invKey[0][1] = (-key[0][1] * detInv) % 26;
    invKey[1][0] = (-key[1][0] * detInv) % 26;
    invKey[1][1] = (key[0][0] * detInv) % 26;

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            if (invKey[i][j] < 0) invKey[i][j] += 26;

    // Decrypt: P = C * K_inverse
    String plain = "";
    for (int i = 0; i < cipher.length(); i += 2) {
        int c1 = cipher.charAt(i) - 'A';
        int c2 = cipher.charAt(i + 1) - 'A';

        int p1 = (c1 * invKey[0][0] + c2 * invKey[1][0]) % 26;
        int p2 = (c1 * invKey[0][1] + c2 * invKey[1][1]) % 26;

        plain += (char) (p1 + 'A');
        plain += (char) (p2 + 'A');
    }
    return plain;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```

```
        // Key [[3,3],[2,5]]
        int[][] key = {{3, 3}, {2, 5}};

        System.out.print("Enter plaintext: ");
        String plain = sc.nextLine();

        String enc = encrypt(plain, key);
        System.out.println("Encrypted: " + enc);

        String dec = decrypt(enc, key);
        System.out.println("Decrypted: " + dec);
    }
}
```

LAB 4: WRITE A JAVA PROGRAM TO PERFORM ENCRYPTION AND DECRYPTION USING RAIL FENCE TRANSPOSITION TECHNIQUE

```
import java.util.*;
class RailFenceCipher {
    static String encrypt(String t, int r){
        StringBuilder[] a = new StringBuilder[r];
        for(int i=0;i<r;i++) a[i]=new StringBuilder();
        int row=0,d=1;
```

```
    for(char c:t.toCharArray()){
        a[row].append(c);
        if(row==0) d=1; else if(row==r-1) d=-1;
        row+=d;
    }
    StringBuilder s=new StringBuilder();
    for(var x:a) s.append(x);
    return s.toString();
}
static String decrypt(String c, int r){
    int n=c.length(); char[][] f=new char[r][n];
    int row=0,d=1;
    for(int i=0;i<n;i++)
    { f[row][i]='*';   if(row==0) d=1;   else if(row==r-1)   d=-1;   row+=d; }
    int k=0;
    for(int i=0;i<r;i++) for(int j=0;j<n;j++)  if(f[i][j]=='*') f[i][j]=c.charAt(k++);
    StringBuilder s=new StringBuilder();  row=0; d=1;
    for(int i=0;i<n;i++)
   { s.append(f[row][i]); if(row==0) d=1; else if(row==r-1) d=-1; row+=d; }
    return s.toString();
}
public static void main(String[] a){
    Scanner sc=new Scanner(System.in);
    System.out.print("Text: "); String t=sc.nextLine();
    System.out.print("Rails: "); int r=sc.nextInt();
```

```
        String e=encrypt(t,r);
        System.out.println("Encrypted: "+e);
        System.out.println("Decrypted: "+decrypt(e,r));
    }
}
```

LAB 5: WRITE A JAVA PROGRAM TO IMPLEMENT SINGLE AND DOUBLE TRANSPOSITION TECHNIQUE

```
import java.util.*;
public class ColumnarTranspositionCipher {

    static int[] getKey(String keykey) {
        String k = keykey.toUpperCase();
        int n = k.length();
        int[] key = new int[n];
        Integer[] idx = new Integer[n];
        for (int i = 0; i < n; i++) idx[i] = i;
        Arrays.sort(idx, (a, b) -> k.charAt(a)!= k.charAt(b)? k.charAt(a) - k.charAt(b) : a
- b);
        for (int r = 0; r < n; r++) key[idx[r]] = r + 1;
        return key;
    }

    static String encrypt(String pt, int[] key) {
        int c = key.length, r = (int) Math.ceil((double) pt.length() / c);
```

```
    char[][] g = new char[r][c];
    for (int i = 0; i < r * c; i++) g[i / c][i % c] = i < pt.length()? pt.charAt(i) : '_';
    StringBuilder sb = new StringBuilder();
    for (int k = 1; k <= c; k++)
        for (int j = 0; j < c; j++) if (key[j] == k)
            for (int i = 0; i < r; i++) sb.append(g[i][j]);
    return sb.toString();
}

static String decrypt(String ct, int[] key) {
    int c = key.length, r = ct.length() / c;
    char[][] g = new char[r][c];
    int p = 0;
    for (int k = 1; k <= c; k++)
        for (int j = 0; j < c; j++) if (key[j] == k)
            for (int i = 0; i < r; i++) g[i][j] = ct.charAt(p++);
    StringBuilder sb = new StringBuilder();
    for (char[] row : g) for (char ch : row) sb.append(ch);
    return sb.toString();
}

// Double Transposition
static String doubleEncrypt(String pt, int[] key) {
    return encrypt(encrypt(pt, key), key);
}
```

```
  static String doubleDecrypt(String ct, int[] key) {
    return decrypt(decrypt(ct, key), key);
  }

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Plaintext: ");
    String msg = sc.nextLine().replace(" ", "");
    System.out.print("Key: ");
    String skey = sc.next();

    int[] key = getKey(skey);
    System.out.println("\nNumeric Key: " + Arrays.toString(key));

    String singleEnc = encrypt(msg, key);
    System.out.println("\n--- Single Transposition ---");
    System.out.println("Encrypted: " + singleEnc);
    System.out.println("Decrypted: " + decrypt(singleEnc, key));

    String doubleEnc = doubleEncrypt(msg, key);
    System.out.println("\n--- Double Transposition ---");
    System.out.println("Encrypted: " + doubleEnc);
    System.out.println("Decrypted: " + doubleDecrypt(doubleEnc,
key).replace("_",""));
  }
```

}

LAB 6: WRITE A JAVA PROGRAM TO IMPLEMENT DES ALGORITHM LOGIC

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.DESKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class DESWithUserKey {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plaintext: ");
        String plainText = sc.nextLine();

        System.out.print("Enter Key (must be 8 chars e.g. 12345678): ");
        String keyInput = sc.nextLine();

        // 1. Make key from user input string
        DESKeySpec desKeySpec = new DESKeySpec(keyInput.getBytes());
        SecretKeyFactory keyFactory = SecretKeyFactory.getInstance("DES");
```

```
        SecretKey secretKey = keyFactory.generateSecret(desKeySpec);


        // 2. Encrypt
        Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);
        byte[] encryptedBytes = cipher.doFinal(plainText.getBytes());
        String encryptedText =
Base64.getEncoder().encodeToString(encryptedBytes);


        System.out.println("Encrypted: " + encryptedText);


        // 3. Decrypt
        cipher.init(Cipher.DECRYPT_MODE, secretKey);
        byte[] decryptedBytes =
cipher.doFinal(Base64.getDecoder().decode(encryptedText));
        String decryptedText = new String(decryptedBytes);


        System.out.println("Decrypted: " + decryptedText);
    }
}
```

LAB 7: WRITE A JAVA PROGRAM TO IMPLEMENT AES ALGORITHM LOGIC

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
```

```
import java.util.Scanner;

public class AES {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter 16 char Key: ");
        String key = sc.nextLine();

        System.out.print("Enter Plain Text: ");
        String plain = sc.nextLine();

        // Encrypt
        SecretKeySpec secKey = new SecretKeySpec(key.getBytes(), "AES");
        Cipher cipher = Cipher.getInstance("AES");

        cipher.init(Cipher.ENCRYPT_MODE, secKey);
        String encrypted =
Base64.getEncoder().encodeToString(cipher.doFinal(plain.getBytes()));

        // Decrypt
        cipher.init(Cipher.DECRYPT_MODE, secKey);
        byte[] decoded = Base64.getDecoder().decode(encrypted);
        String decrypted = new String(cipher.doFinal(decoded));
```

```
        System.out.println("\nEncrypted : " + encrypted);
        System.out.println("Decrypted : " + decrypted);

        sc.close();
    }
}
```

LAB 8: WRITE A JAVA PROGRAM TO IMPLEMENT TRIPLE DES ALGORITHM LOGIC

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class TripleDES {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter 24 char Key: ");
        String key = sc.nextLine();

        System.out.print("Enter Plain Text: ");
        String plain = sc.nextLine();

        // Triple DES needs 24 byte key
```

```
        SecretKeySpec secKey = new SecretKeySpec(key.getBytes(), "DESede");
        Cipher cipher = Cipher.getInstance("DESede"); //
DESede/ECB/PKCS5Padding


        // Encrypt
        cipher.init(Cipher.ENCRYPT_MODE, secKey);
        String encrypted =
Base64.getEncoder().encodeToString(cipher.doFinal(plain.getBytes()));


        // Decrypt
        cipher.init(Cipher.DECRYPT_MODE, secKey);
        String decrypted = new
String(cipher.doFinal(Base64.getDecoder().decode(encrypted)));


        System.out.println("\nEncrypted : " + encrypted);
        System.out.println("Decrypted : " + decrypted);


        sc.close();
    }
}
```

LAB 9: WRITE A JAVA PROGRAM TO IMPLEMENT RC4 ALGORITHM LOGIC, ENCRYPT “HELLO WORLD”

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
```

```
import java.util.Base64;


public class RC4 {
   public static void main(String[] args) throws Exception {
      String plain = "Hello world";


      KeyGenerator kg = KeyGenerator.getInstance("ARCFOUR");
      kg.init(128); //40 to 2048
      SecretKey key = kg.generateKey();


      Cipher c = Cipher.getInstance("ARCFOUR");


      c.init(Cipher.ENCRYPT_MODE, key);
      String enc =
Base64.getEncoder().encodeToString(c.doFinal(plain.getBytes()));


      c.init(Cipher.DECRYPT_MODE, key);
      String dec = new String(c.doFinal(Base64.getDecoder().decode(enc)));


      System.out.println("Key: " +
Base64.getEncoder().encodeToString(key.getEncoded()));
      System.out.println("Plain: " + plain);
      System.out.println("Enc: " + enc);
      System.out.println("Dec: " + dec);
   }
```

}

LAB 10: WRITE A JAVA PROGRAM TO IMPLEMENT RSA ALGORITHM LOGIC

```
import javax.crypto.Cipher;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.util.Base64;
import java.util.Scanner;

public class RSA {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Plain Text: ");
        String plain = sc.nextLine();
// 1. Generate Key Pair -1024, 2048, 3072 or 4096 bits.
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
        keyGen.initialize(2048);
        KeyPair pair = keyGen.generateKeyPair();
 // 2. Encrypt with Public Key
        Cipher cipher = Cipher.getInstance("RSA");
        cipher.init(Cipher.ENCRYPT_MODE, pair.getPublic());
        String encrypted =
Base64.getEncoder().encodeToString(cipher.doFinal(plain.getBytes()));
 // 3. Decrypt with Private Key
        cipher.init(Cipher.DECRYPT_MODE, pair.getPrivate());
```

```
        String decrypted = new
String(cipher.doFinal(Base64.getDecoder().decode(encrypted)));

        System.out.println("\nEncrypted : " + encrypted);

        System.out.println("Decrypted : " + decrypted);


        sc.close();

    }

}
```

LAB 11: PROGRAM TO IMPLEMENT DIFFIE HELLMAN USING JAVASCRIPT AND HTML

```
<h2>Diffie-Hellman</h2>

<label>p: <input id="p" title="p" placeholder="prime" value="23"></label>

<label>g: <input id="g" title="g" placeholder="generator" value="5"></label>

<label>a: <input id="a" title="a" placeholder="alice private" value="6"></label>

<label>b: <input id="b" title="b" placeholder="bob private" value="15"></label>

<button onclick="go()">Go</button>

<p id="r"></p>


<script>

function pow(m,e,n)

{let r=1; m%=n;

    while(e>0)

    {if(e%2) r=r*m%n;

        e>>=1;

        m=m*m%n
```

```
    }
    return r
 }
function isPrime(n){
    if(n<2) return false;
    if(n==2) return true;
    if(n%2==0) return false;
    for(let i=3;i*i<=n;i+=2) if(n%i==0) return false;
    return true;
}
// Check if g is primitive root of p
function isPrimitiveRoot(g, p) {
    if (g <= 1 || g >= p) return false;
    let phi = p - 1;
    let factors = [];
    let n = phi;
    // find prime factors of phi
    for (let i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            factors.push(i);
            while (n % i == 0) n /= i;
        }
    }
    if (n > 1) factors.push(n);
    for (let f of factors) {
```

```
    if (pow(g, phi / f, p) == 1) return false;
  }
  return true;
}

function go(){
let p=+document.getElementById('p').value,
  g=+document.getElementById('g').value,
  a=+document.getElementById('a').value,
  b=+document.getElementById('b').value;
let out = document.getElementById('r');

if (!isPrimitiveRoot(g, p)) {
  document.getElementById('r').innerHTML = `<b style="color:red">Error: ${g} is NOT a primitive root of ${p}</b>`;
  return;
}
if (!isPrime(p)) {
  out.innerHTML = `<b style="color:red">Error: p=${p} is NOT prime. Enter a prime number.</b>`;
  return;
}

let A1=pow(g,a,p), B1=pow(g,b,p);
let s1=pow(B1,a,p), s2=pow(A1,b,p);
document.getElementById('r').innerHTML=`g=${g} is valid primitive root of p=${p} ✅<br>A=${A1} B=${B1}<br>Secret=${s1} ${s1==s2?'Matched':'Failed'}`;
```

```
}
```

LAB 12: PROGRAM TO CALCULATE MESSAGE DIGEST OF A TEXT USING SHA 1 ALGORITHM IN JAVA

```
import java.security.MessageDigest;
import java.util.Scanner;

public class SHA1Digest {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String message = sc.nextLine();

        MessageDigest md = MessageDigest.getInstance("SHA-1");
        byte[] hashBytes = md.digest(message.getBytes("UTF-8"));

        StringBuilder hexString = new StringBuilder();
        for (byte b : hashBytes) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }

        System.out.println("SHA-1 Digest: " + hexString.toString());
        sc.close();
```

```
    }
}
```

LAB 13: PROGRAM TO CALCULATE MESSAGE DIGEST OF A TEXT USING MD-5 ALGORITHM IN JAVA

```
import java.security.MessageDigest;
import java.util.Scanner;

public class MD5Digest {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter message: ");
        String message = sc.nextLine();

        MessageDigest md = MessageDigest.getInstance("MD5");
        byte[] hashBytes = md.digest(message.getBytes("UTF-8"));

        StringBuilder hexString = new StringBuilder();
        for (byte b : hashBytes) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }

        System.out.println("MD-5 Digest: " + hexString.toString());
```

```
        sc.close();
    }
}
```

TOPIC TO LEARN FOR VIVA (As it is not there in theory syllabus)

MD5 (Message-Digest Algorithm 5) is a cryptographic hash function developed by **Ronald Rivest** in **1991** to replace the insecure MD4 algorithm. It processes variable-length input data into a fixed **128-bit** (16-byte) hash value, typically represented as a **32-character hexadecimal string**.

The algorithm operates by breaking input into **512-bit blocks**, padding them to fit, and processing them through **four rounds** of 16 operations each using bitwise functions and modular addition. While originally designed for digital signatures and message authentication, MD5 is now considered **cryptographically broken** due to demonstrated vulnerabilities to collision attacks.

Key Characteristics:

- **Output:** 128-bit hash (32 hex characters).
- **Structure:** Merkle–Damgård construction with 64 total operations.
- **Status:** Insecure for cryptographic purposes (e.g., password storage, digital signatures); **RFC 6151** recommends against its use where collision resistance is required.
- **Current Use:** Still widely used for **non-cryptographic** purposes like file integrity checksums and data deduplication in pipelines.
- **Alternatives:** For security-sensitive applications, modern algorithms like **SHA-256**, **bcrypt**, or **Argon2** are preferred.

LAB 14: Implement the SIGNATURE SCHEME – Digital Signature Standard in Java

```
import java.security.*;

public class DSSExample {
```

```
public static void main(String[] args) throws Exception {
    String message = "hello";

    // 1. Generate DSA Key Pair - DSS requires DSA keys (not RSA)
    KeyPairGenerator keyGen = KeyPairGenerator.getInstance("DSA");
    keyGen.initialize(2048); // DSS standard key size
    KeyPair keyPair = keyGen.generateKeyPair();

    // 2. Sign with Private Key
    // DSS uses SHA256withDSA (modern version of SHA1withDSA)
    Signature dsaSign = Signature.getInstance("SHA256withDSA");
    dsaSign.initSign(keyPair.getPrivate());
    dsaSign.update(message.getBytes());
    byte[] signature = dsaSign.sign();

    System.out.println("Message: " + message);
    System.out.println("DSS Signature generated");

    // 3. Verify with Public Key
    dsaSign.initVerify(keyPair.getPublic());
    dsaSign.update(message.getBytes());
    boolean verified = dsaSign.verify(signature);
```

```
        System.out.println("Verified: " + verified);


        // 4. Tamper test
        dsaSign.initVerify(keyPair.getPublic());
        dsaSign.update("tampered message".getBytes());
        System.out.println("Tampered Verified: " + dsaSign.verify(signature));
    }
}
```